

# Design Document for "Better flashcards"

---

## Content

- 1. General information
- 2. Project description
- 3. Objectives
- 4. Features
- 3. Technologies
- 3. Architecture
- 3. Possible Future Developments

## 1. General Information

- **Project name:** Better Flashcards
- **Team members:** Maksym Makovskyi
- **Git repository link:** <https://github.com/maxmakovskiy/better-flashcards.git>

## 2. Project Description

It is a learning platform that uses flashcards in its core, but it also provides tools to analyze learning progress. It is something that I personally use very often, primary example is Anki flashcards, so it is really easy to concretize the final product.

## 3. Objectives

The platform allows users to:

- Create and organize personalized learning materials into structured study decks
- Improve long-term knowledge retention through adaptive repetition and review scheduling
- Track learning progress and identify weak areas over time

## 4. Features

### 4.1 Core Features

The features the application **must** provide in order to be considered complete:

- User authentication with identity provider
- Deck and Flashcard management (CRUD decks/cards, cards include text)
- Adaptive study system (spaced repetition scheduling, ~~SM-2~~ [FSRS](#))
- Interactive study mode (guess the other side of a card)
- Learning dashboard (track study progress, view study statistics)

### 4.2 Optional Features

Bonus features if time permits:

- Cards can include the images
- More study modes (pair it/choose it/type it)
- Knowledge Graph Visualization of related cards within (among) subject(s)
- Replay previous study session
- Shared decks and collaborative editing
- Public/private decks

## 5. Technologies

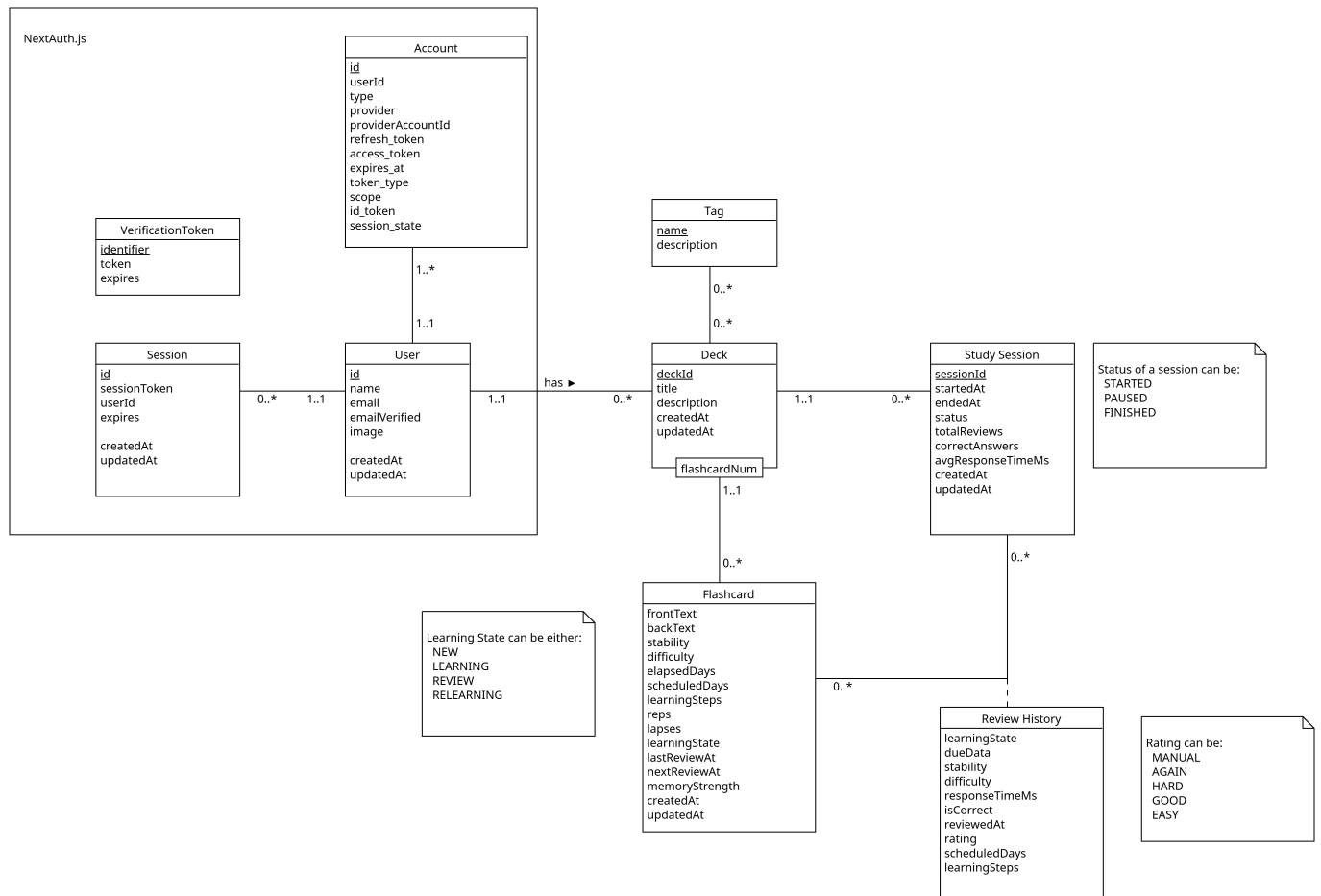
List the planned technologies and **briefly justify each choice**.

For example:

- **Frontend:**
  - React (DOM manipulating lib)
  - MUI (component lib)
  - Zustand (state management, if React Context is not enough)
  - ~~Motion for React (card animations)~~ (it does not follow material design guideline)
  - SWR (simple fetch and state management things)
  - Zod (ensuring data correctness in client-server exchange)
  - FSRS or Free Spaced Repetition Scheduler (spaced repetition algorithm)
  - Date-fns (date manipulating and formatting)
- **Backend:** Next.js with Typescript
- **Database:** Prisma (orm), Sqlite (database)
- **Authentication:** NextAuth.js

## 6. Architecture

Overview of the components and their interactions (client, server, database, third-party services, etc.). A small diagram is welcome.



## 7. Possible Future Developments

(Optional) Ideas for future improvements identified but outside the scope of the lab/project:

- Elements of social network (list of friends/clubs of interest/competitions)